

PROJECT REPORT

Smart Student Management & Attendance System

COURSE DETAILS

Course Name: Programming and Problem Solving Lab
Semester: Spring 2026 | **Section:** 70s (s2)

Submitted To:

MD Taufik Hasan
Lecturer
Department of Computer Science
and Engineering
Daffodil International University

Submitted By (Team Members):

Ayan Majumder	253-15-158
Kabya Ghosh	252-15-254
MD Mostafezur Rahaman	253-15-549
Soumitro Day	252-15-522
MD. Masudur Rahman	253-15-486

Date: April 22, 2026

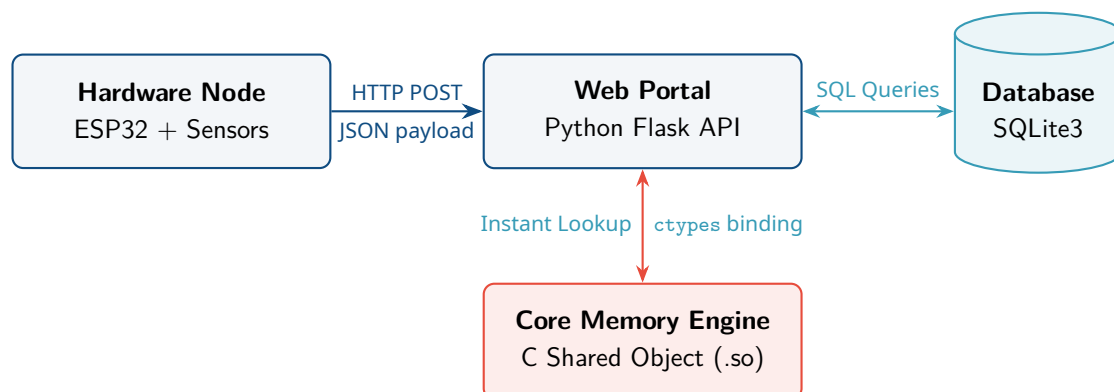
1. Abstract

The Smart Student Management & Attendance System is an Internet of Things (IoT) based project designed to automate the academic attendance process. Traditional attendance systems are manual, time-consuming, and prone to human error. This project provides a robust solution by integrating an ESP32 hardware scanner (utilizing Fingerprint and RFID technology) with a Python (Flask) web portal.

A key innovation in this project is the integration of a custom-built, high-speed C-language memory engine that interacts directly with the server's RAM, providing instantaneous student verification without database lag. The system also features an automated SMS notification system for parents and an SD-card backed offline mode for resilience against network failures.

2. System Architecture

The project architecture is built on a three-tier model, separating hardware collection, backend logic, and high-speed memory optimization.



- A. The Hardware Node (Client):** Collects biometric and RFID data, operating asynchronously.
- B. The Web Portal (Application Layer):** Provides the GUI, handles persistent storage, and runs background threads (e.g., SMS APIs).
- C. The Core Memory Engine (Optimization):** Bypasses standard DB lookup latency during mass classroom entries.

3. Hardware Implementation & Specifications

The physical scanner unit is stationed at the classroom entrance. It requires low latency and high reliability. The technical breakdown of the node is detailed in Table 1.

4. Software Implementation & Technical Details

Table 1: Hardware Components and Technical Specifications

Component	Model	Technical Function	Interface
Microcontroller	ESP32-WROOM	Core logic, Wi-Fi 802.11 b/g/n, and task scheduling (FreeRTOS).	N/A
Biometric Sensor	AS608 Optical	Secure fingerprint imaging and local template matching.	UART (9600)
Proximity Sensor	PN532 Module	High-speed 13.56MHz RFID card reading.	I2C
Display Unit	0.96" OLED	Real-time GUI feedback (128x64 pixels).	I2C
Local Storage	MicroSD Adapter	Caching whitelist arrays and offline attendance logs (.txt/.json).	SPI

4.1 The High-Speed C Engine (Performance Optimization)

To eliminate latency during mass scanning, a core engine was written in C. Standard databases (like SQLite) incur disk I/O latency. Instead, our C engine allocates a fixed block of RAM on server boot.

Below is the internal data structure used by the C engine to maintain synchronization with the Python application:

Listing 1: Core Memory Engine Struct (C Language)

```
#include <stdint.h>

// Fixed-size structure allocated in RAM for zero-latency lookups
typedef struct {
    char rfid_uid[16];           // 14-char Hex String + Null Terminator
    uint16_t fingerprint_id;    // 1-127 integer mapped to AS608 scanner
    char student_name[64];      // Pre-fetched name for instant OLED display
    uint8_t is_present;         // Boolean flag (0 or 1) for the current session
} StudentRecord;

// Global Array Pointer (Managed via Python ctypes)
StudentRecord* active_session_cache;
```

When an RFID tag is scanned, Python passes the string to the C library, which uses hardware-accelerated `strcmp()` or memory-aligned comparisons to perform an instant match and return the student data back to the Flask API.

4.2 Communication Protocol (API Design)

The ESP32 communicates with the Flask server via RESTful HTTP endpoints. Data is serialized into lightweight JSON to minimize bandwidth and processing time on the microcontroller.

Listing 2: Example Payload from ESP32 to Flask Server

```
POST /api/v1/attendance/scan HTTP/1.1
Host: server.local:5000
Content-Type: application/json

{
  "device_id": "ESP_ROOM_402",
  "scan_type": "RFID",
  "uid": "A4B9C210",
  "timestamp": 1715428900,
  "offline_sync": false
}
```

5. Advanced Functionalities

5.1 Automated SMS Notifications

The system runs background threads utilizing standard telecom APIs. If the portal is manually "closed" by the teacher at the end of the day, the system executes an array-difference operation: comparing the full enrolled list against the `is_present` flags in the C-Engine. An absence SMS is automatically pushed to the parent contacts of unflagged students.

5.2 Offline Resilience & Batch Sync (FreeRTOS)

If the Wi-Fi network drops, the system defaults to **Resilience Mode**:

- **Local Verification:** The ESP32 checks the scanned ID against a pre-downloaded whitelist on the MicroSD card.
- **Data Queuing:** It logs the attendance timestamp locally.
- **Auto-Recovery:** A FreeRTOS background task continuously pings the gateway. Upon connection restoration, it bundles the saved logs into a JSON array and batch-syncs them to the server, ensuring zero data loss.

6. Conclusion

The Smart Student Management & Attendance System successfully demonstrates how integrating low-level memory management (C) with high-level web frameworks (Python Flask) and IoT hardware (ESP32) yields a commercial-grade product. By optimizing the "hot path" of data scanning through RAM, it completely eliminates the bottleneck of manual roll calls, keeps parents informed in real-time, and guarantees high availability through offline caching.